



Linear Regression — Code Documentation

Boston Housing Price Prediction · scikit-learn

A line-by-line walkthrough of a Python script that loads the Boston housing dataset, trains a Linear Regression model to predict house prices, and grades the result using cross-validation and the R^2 , MAE and RMSE error metrics. Read the full program first, then follow each step with a plain-English explanation beside the code.



pandas



numpy



scikit-learn



matplotlib / seaborn

FILE

linear_regression.py

LANGUAGE

Python 3

STEPS

11 explained blocks

Before the Code: Key Concepts

theory + practical

This project leans on three core ideas from machine learning. Here is each one in plain language — what it means and how it shows up in the script — so the line-by-line walkthrough that follows makes complete sense.



1 · What is Linear Regression?

THE MODEL

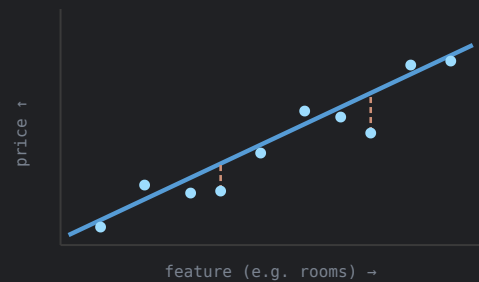
THE IDEA

Linear regression is one of the simplest ways for a computer to learn from data. Picture all your data plotted as dots on a graph, then draw the *single straight line* that passes as close as possible through the middle of them. That line becomes a **prediction rule**: give it an input, read off the output. With many inputs (number of rooms, crime rate, distance to work...) the model simply **weighs each one** — adding value for good features, subtracting for bad — to arrive at a final number. "Training" just means nudging those weights until the line sits as close as possible to every real point.

$$\text{price} \approx w_1 \cdot \text{rooms} + w_2 \cdot \text{crime} + \dots + b$$

IN THIS CODE

`LinearRegression()` creates the empty rule, `.fit(X_train, Y_train)` learns the best weights from example houses, and `.predict(X_test)` uses them to estimate prices for houses it has never seen.



Dots = real houses · blue line = model's best fit
orange = the prediction error for a house



2 · What is Cross-Validation?

THE TEST

THE IDEA

How do you know a model is genuinely good and not just *lucky* on one test? Cross-validation checks it several times on different slices of the data. The rows are cut into equal parts called **folds**. The model trains on all-but-one fold and is tested on the fold left out — then it rotates, so every fold takes a turn as the test set. Averaging all those scores gives a far more **trustworthy** verdict than a single split, because the result no longer depends on which rows happened to be tested.

IN THIS CODE

`cross_val_score(..., cv=5)` runs this 5 times (5 folds) and returns 5 scores; `np.mean` averages them into one summary number.



5-fold cross-validation: each run tests on a different slice, trains on the rest

Why the score looks negative: scikit-learn always assumes "higher = better," but error should be *lower*. So for `scoring="neg_mean_squared_error"` it reports the error as a **negative** number — a value closer to 0 means a better model. Flip the sign to read the real error.



3 · The Error Metrics (R^2 , MAE, RMSE)

THE SCORE

THE IDEA

Once the model makes predictions, we need to measure **how wrong** it is. Each metric does that in a slightly different way:

Metric	What it tells you (in plain words)	Better when	Units
R^2	The share of the price pattern the model manages to explain. 1 = perfect; 0 = no better than always guessing the average price.	closer to 1	none (0-1)
MAE	The average gap between predicted and real prices. Simple and forgiving — treats every miss equally.	lower	same as price
RMSE	Like MAE, but it squares errors first, so a few <i>big</i> misses hurt the score much more. Always $\geq$ MAE.	lower	same as price

IN THIS CODE

`metrics.r2_score`, `metrics.mean_absolute_error` and `np.sqrt(metrics.mean_squared_error(...))` compute these three on the test set. For this model they come out to roughly $R^2 = 0.66$, **MAE = 3.18** and **RMSE = 4.92** — i.e. it explains about 66% of the price pattern and is off by around 3-5 price units on average.

The Complete Program

linear_regression.py

```
1 import pandas as pd
2 import numpy as np
3 import matplotlib.pyplot as plt
4 import seaborn as sns
5 from sklearn.datasets import fetch_openml
6
7 boston = fetch_openml(name="boston", version=1, as_frame=True, parser="pandas")
8
9 df = boston.frame
10 df["Price"] = boston.target
11
12 X = df.drop(columns=["MEDV", "Price"])
13 Y = df["Price"]
14
15 from sklearn.linear_model import LinearRegression
16 from sklearn.model_selection import cross_val_score
17
18 X = X.astype(float)
19 Y = Y.astype(float)
20 lin_reg = LinearRegression()
21
22 mse = cross_val_score(lin_reg, X, Y, scoring="neg_mean_squared_error", cv=5)
23 mean_mse = np.mean(mse)
24
25 from sklearn.model_selection import train_test_split
26
27 X_train, X_test, Y_train, Y_test = train_test_split(
28     X, Y, train_size=0.8, random_state=42
29 )
30 lin_reg.fit(X_train, Y_train)
31
32 prediction = lin_reg.predict(X_test)
33 comparision_df = pd.DataFrame({"Actual": Y_test, "Predicted": prediction})
34 print(comparision_df.head())
35
36 from sklearn import metrics
37
38 r2 = metrics.r2_score(Y_test, prediction)
39 mae = metrics.mean_absolute_error(Y_test, prediction)
40 rmse = np.sqrt(metrics.mean_squared_error(Y_test, prediction))
41
42 print("R SQUARED ERROR ", r2)
43 print("MEAN ABSOLUTE ERROR", mae)
44 print("ROOT MEAN SQUARED ERROR", rmse)
```

⚠ HEADS UP

The split below uses **train_size=0.8**, which trains on 80% of the data and tests on 20%.

Line-by-Line Walkthrough

[code](#) → [explanation](#)

01 Import the libraries

```
linear_regression.py • Python
1 import pandas as pd
2 import numpy as np
3 import matplotlib.pyplot as plt
4 import seaborn as sns
5 from sklearn.datasets import fetch_openml
```

EXPLANATION

Pulls in every tool the script needs up front. **pandas** handles tables (DataFrames), **numpy** does fast numerical math, and **matplotlib** / **seaborn** are plotting libraries kept ready for later. The final line imports `fetch_openml`, which downloads ready-made datasets from openml.org.

02 Download the dataset

```
linear_regression.py • Python
1 boston = fetch_openml(name="boston", version=1, as_frame=True, parser="pandas")
```

EXPLANATION

Downloads the Boston housing dataset in one call. `name` and `version` pin down *exactly* which dataset to fetch, `as_frame=True` asks for it back as a pandas table instead of raw arrays, and `parser="pandas"` tells scikit-learn to read the downloaded file using pandas.

03 Build the working table

```
linear_regression.py • Python
1 df = boston.frame
2 df["Price"] = boston.target
```

EXPLANATION

`boston.frame` is the complete table — all the feature columns plus the target — so `df` now holds everything. The second line copies the target prices (the `MEDV` column) into a friendlier-named `Price` column, lined up row by row. These are **not** random values; `Price` is simply a renamed copy of `MEDV`.

► Dataset preview – before vs. after adding `Price`

OUTPUT `df.head()` – before adding `Price`

	CRIM	ZN	INDUS	CHAS	NOX	RM	AGE	DIS	RAD	TAX	PTRATIO	B	LSTAT	MEDV
0	0.00632	18	2.31	0	0.538	6.575	65.2	4.09	1	296	15.3	396.9	4.98	24
1	0.02731	0	7.07	0	0.469	6.421	78.9	4.9671	2	242	17.8	396.9	9.14	21.6
2	0.02729	0	7.07	0	0.469	7.185	61.1	4.9671	2	242	17.8	392.83	4.03	34.7
3	0.03237	0	2.18	0	0.458	6.998	45.8	6.0622	3	222	18.7	394.63	2.94	33.4
4	0.06905	0	2.18	0	0.458	7.147	54.2	6.0622	3	222	18.7	396.9	5.33	36.2

The full table from `boston.frame`: 13 feature columns plus the target `MEDV` (the actual median house price).

OUTPUT df.head() — after df["Price"] = boston.target

	CRIM	ZN	INDUS	CHAS	NOX	RM	AGE	DIS	RAD	TAX	PTRATIO	B	LSTAT	MEDV	Price
0	0.00632	18	2.31	0	0.538	6.575	65.2	4.09	1	296	15.3	396.9	4.98	24	24
1	0.02731	0	7.07	0	0.469	6.421	78.9	4.9671	2	242	17.8	396.9	9.14	21.6	21.6
2	0.02729	0	7.07	0	0.469	7.185	61.1	4.9671	2	242	17.8	392.83	4.03	34.7	34.7
3	0.03237	0	2.18	0	0.458	6.998	45.8	6.0622	3	222	18.7	394.63	2.94	33.4	33.4
4	0.06905	0	2.18	0	0.458	7.147	54.2	6.0622	3	222	18.7	396.9	5.33	36.2	36.2

A new **Price** column is appended — notice it is an exact copy of **MEDV**, value for value. That confirms **Price** is just a friendlier-named duplicate, not new data.

04 Separate inputs (X) from the answer (Y)

```
1 X = df.drop(columns=["MEDV", "Price"])
2 Y = df["Price"]
```

Python

EXPLANATION

Splits the table into what the model *sees* and what it must *predict*. **X** keeps every house feature but drops both **MEDV** and its copy **Price** so the model can't peek at the answer. **Y** is that answer — the price we want to predict.

05 Load the model tools and tidy the data

```
1 from sklearn.linear_model import LinearRegression
2 from sklearn.model_selection import cross_val_score
3
4 X = X.astype(float)
5 Y = Y.astype(float)
6 lin_reg = LinearRegression()
```

Python

EXPLANATION

Imports the **LinearRegression** model and the **cross_val_score** helper used to grade it. The two **.astype(float)** calls force every value to a decimal number so the math behaves, and the last line creates an empty, untrained model object called **lin_reg**.

06 Quick quality check with cross-validation

```
1 mse = cross_val_score(lin_reg, X, Y, scoring="neg_mean_squared_error", cv=5)
2 mean_mse = np.mean(mse)
```

Python

EXPLANATION

cv=5 chops the data into 5 folds, trains on 4 and tests on the leftover 1, repeating until every fold has been the test set once. scikit-learn always assumes *higher = better*, but error should be *lower*, so it returns the **negative** MSE — a value closer to 0 means a better model. **np.mean** then averages the 5 scores into one number.

07 Split into training and testing sets

```
linear_regression.py • Python
1 from sklearn.model_selection import train_test_split
2
3 X_train, X_test, Y_train, Y_test = train_test_split(
4     X, Y, train_size=0.8, random_state=42
5 )
```

EXPLANATION

Carves the data into a set to learn from and a set to be tested on. `random_state=42` fixes the random shuffle so you get the same split on every run. `train_size=0.8` trains on **80%** of the rows and tests on the other 20% `test_size=0.2`).

08 Train the model

```
linear_regression.py • Python
1 lin_reg.fit(X_train, Y_train)
```

EXPLANATION

This is the actual learning step. `.fit()` looks at the training features and prices together and works out the best-fit straight line (the coefficients) that maps house features to price.

09 Predict on unseen data and compare

```
linear_regression.py • Python
1 prediction = lin_reg.predict(X_test)
2 comparision_df = pd.DataFrame({"Actual": Y_test, "Predicted": prediction})
3 print(comparision_df.head())
```

EXPLANATION

Feeds the held-out test features into the trained model to get predicted prices, builds a small table placing the **actual** prices next to the **predicted** ones, and prints the first few rows so you can eyeball how close the model got.

► Console output – first rows of actual vs. predicted

```
TERMINAL
PS> python linear_regression.py

Actual Predicted
173      23.6    28.996724
274      32.4    36.025567
491      13.6    14.816944
72       22.8    25.031979
452      16.1    18.769880
```

Indices (173, 274, ...) are the original row numbers that landed in the test set. Predictions land within a few thousand dollars of the real prices.

10 Score the model with error metrics

Python

```
1 from sklearn import metrics
2
3 r2 = metrics.r2_score(Y_test, prediction)
4 mae = metrics.mean_absolute_error(Y_test, prediction)
5 rmse = np.sqrt(metrics.mean_squared_error(Y_test, prediction))
```

EXPLANATION

Three standard yardsticks. **R²** is the share of price variation the model explains (closer to 1 is better). **MAE** is the average error size in price units (lower is better). **RMSE** is similar but punishes big misses harder; the `np.sqrt` brings it back into normal price units.

11 Print the results

Python

```
1 print("R SQUARED ERROR ", r2)
2 print("MEAN ABSOLUTE ERROR", mae)
3 print("ROOT MEAN SQUARED ERROR", rmse)
```

EXPLANATION

Displays the three scores in the terminal so you can read off, at a glance, how well the model performed on the test data.

> Console output – final model scores

TERMINAL

```
PS> python linear_regression.py

R SQUARED ERROR    0.6687594935356317
MEAN ABSOLUTE ERROR 3.1890919658878416
ROOT MEAN SQUARED ERROR 4.928602182665339
```

$R^2 \approx 0.66$ means the model explains about 66% of the price variation. On average it is off by about **3.18** (MAE) price units, with an RMSE of **4.92** — respectable given it trained on only 80% of the data.